

Exercise 8 – HTTP Client and REST API (GET)

Install Required Packages

npm install

npm install json-server

Create db.json

```
{
  "courses":[
    {
      "id":1,
      "name":"Angular",
      "code":"ANG101",
      "credits":4,
      "faculty":"Mr. Kumar"
    },
    {
      "id":2,
      "name":"React",
      "code":"RCT201",
      "credits":3,
      "faculty":"Mrs. Priya"
    },
    {
      "id":3,
      "name":"Spring Boot",
      "code":"SPR301",
      "credits":5,
      "faculty":"Mr. Raj"
    }
  ]
}
```

```
},
{
  "id":4,
  "name":"Java",
  "code":"JAVA101",
  "credits":4,
  "faculty":"Mr. Arun"
},
{
  "id":5,
  "name":"MySQL",
  "code":"SQL101",
  "credits":2,
  "faculty":"Mrs. Kavya"
}
]
```

Start JSON Server

```
npx json-server --watch db.json --port 3000
```

app.config.ts

```
import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient } from '@angular/common/http';
import { routes } from './app.routes';

export const appConfig: ApplicationConfig = {
```

```
providers:[
  provideRouter(routes),
  provideHttpClient()
]
};
```

course.model.ts

```
export interface Course{
  id:number;
  name:string;
  code:string;
  credits:number;
  faculty:string;
}
```

course.service.ts

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { Course } from '../models/course.model';
@Injectable({
  providedIn:'root'
})
export class CourseService{
  private apiUrl='http://localhost:3000/courses';
  constructor(private http:HttpClient){}
  getCourses():Observable<Course[]>{
    return this.http.get<Course[]>(this.apiUrl);
```

```
}  
getCourse(id:number):Observable<Course>{  
  return this.http.get<Course>(` ${this.apiUrl}/${id}` );  
}  
}
```

course-list.component.ts

```
import { Component, OnInit } from '@angular/core';  
import { CourseService } from '../services/course.service';  
import { Course } from '../models/course.model';  
import { CourseCardComponent } from '../components/course-card/course-card.component';  
  
@Component({  
  selector:'app-course-list',  
  standalone:true,  
  imports:[CourseCardComponent],  
  templateUrl:'./course-list.component.html',  
  styleUrls:['./course-list.component.css']  
})  
export class CourseListComponent implements OnInit{  
  courses:Course[]=[];  
  constructor(private courseService:CourseService){}  
  ngOnInit(){  
    this.loadCourses();  
  }  
  loadCourses(){  
    this.courseService.getCourses().subscribe({
```

```
next:data=>{
  this.courses=data;
},
error:error=>{
  console.log(error);
}
});
}

selectedCourse(id:number){
  alert('Selected Course ID : '+id);
}
}
```

course-list.component.html

```
<h2>Available Courses</h2>
<app-course-card
  *ngFor="let course of courses"
  [course]="course"
  (enrollRequested)="selectedCourse($event)">
</app-course-card>
```

course-card.component.ts

```
import { Component, EventEmitter, Input, Output } from '@angular/core';
@Component({
  selector:'app-course-card',
  standalone:true,
  templateUrl:'./course-card.component.html',
  styleUrls:['./course-card.component.css']
})
```

```
})  
  
export class CourseCardComponent{  
  @Input()  
  course:any;  
  @Output()  
  enrollRequested=new EventEmitter<number>();  
  enroll(){  
    this.enrollRequested.emit(this.course.id);  
  }  
}
```

course-card.component.html

```
<div class="card">  
  <h3>{{course.name}}</h3>  
  <p>Course Code : {{course.code}}</p>  
  <p>Credits : {{course.credits}}</p>  
  <p>Faculty : {{course.faculty}}</p>  
  <button (click)="enroll()">  
    Enroll  
  </button>  
</div>
```

course-card.component.css

```
.card{  
  padding:20px;  
  margin:15px;  
  border:1px solid #ccc;  
  border-radius:8px;
```

```
box-shadow:0 2px 4px rgba(0,0,0,.2);  
}
```

Error Handling Example

```
this.courseService.getCourses().subscribe({  
  next:data=>{  
    this.courses=data;  
  },  
  error:error=>{  
    alert('Unable to Load Courses');  
    console.log(error);  
  }  
});
```

Run Angular

```
ng serve
```

Run JSON Server

```
npx json-server --watch db.json --port 3000
```

Open Browser

```
http://localhost:4200
```

Output Screenshot:

1. JSON Server Running on http://localhost:3000

2. Course List Loaded from REST API

3. After Clicking Enroll (Course ID 3)

4. Console Logs (API Call & Response)

5. Error Handling (API Unavailable)

Key Outcomes Demonstrated

- ✓ JSON Server running on http://localhost:3000
- ✓ Angular retrieves data using HttpClient GET
- ✓ Course list displayed from REST API
- ✓ Enroll button sends course ID to parent component
- ✓ Errors handled gracefully
- ✓ Console logs show API interactions
- ✓ Successful integration between Angular and REST API using HttpClient

HTTP Client and REST API (POST, PUT, DELETE, Interceptors)

Create HTTP Interceptor

ng generate interceptor interceptors/auth

auth.interceptor.ts

```
import { HttpInterceptorFn } from '@angular/common/http';

export const authInterceptor: HttpInterceptorFn = (req, next) => {

  const modifiedRequest=req.clone({

    setHeaders:{

      Authorization:'Bearer angular-token'

    }

  });

  return next(modifiedRequest);

};
```


app.config.ts

```
import { ApplicationConfig } from '@angular/core';
import { provideRouter } from '@angular/router';
import { provideHttpClient,withInterceptors } from '@angular/common/http';
import { routes } from './app.routes';
import { authInterceptor } from './interceptors/auth.interceptor';

export const appConfig:ApplicationConfig={
  providers:[
    provideRouter(routes),
    provideHttpClient(withInterceptors([authInterceptor]))
  ]
};
```

course.model.ts

```
export interface Course{
  id:number;
  name:string;
  code:string;
  credits:number;
  faculty:string;
}
```

course.service.ts

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable,retry,tap } from 'rxjs';
import { Course } from '../models/course.model';
```

```

@Injectables({
  providedIn:'root'
})

export class CourseService{

  private apiUrl='http://localhost:3000/courses';

  constructor(private http:HttpClient){}

  getCourses():Observable<Course[]>{
    return this.http.get<Course[]>(this.apiUrl).pipe(
      retry(2),
      tap(()=>console.log('Courses Loaded'))
    );
  }

  addCourse(course:Course):Observable<Course>{
    return this.http.post<Course>(this.apiUrl,course);
  }

  updateCourse(course:Course):Observable<Course>{
    return this.http.put<Course>(` ${this.apiUrl}/${course.id}` ,course);
  }

  deleteCourse(id:number):Observable<void>{
    return this.http.delete<void>(` ${this.apiUrl}/${id}` );
  }
}

```

course-list.component.ts

```

import { Component,OnInit } from '@angular/core';
import { CourseService } from '../services/course.service';
import { Course } from '../models/course.model';

```

```
@Component({
  selector:'app-course-list',
  standalone:true,
  templateUrl:'./course-list.component.html',
  styleUrls:['./course-list.component.css']
})
export class CourseListComponent implements OnInit{

  courses:Course[]=[];
  constructor(private courseService:CourseService){}
  ngOnInit(){
    this.loadCourses();
  }
  loadCourses(){
    this.courseService.getCourses().subscribe(data=>{
      this.courses=data;
    });
  }
  addCourse(){
    const course:Course={
      id:6,
      name:'Python',
      code:'PY101',
      credits:3,
      faculty:'Mr. David'
    };
    this.courseService.addCourse(course).subscribe(()=>{
      this.loadCourses();
    });
  }
}
```

```

});
}
updateCourse(){
const course:Course={
id:1,
name:'Angular Advanced',
code:'ANG101',
credits:5,
faculty:'Mr. Kumar'
};
this.courseService.updateCourse(course).subscribe(()=>{
this.loadCourses();
});
}
deleteCourse(id:number){
this.courseService.deleteCourse(id).subscribe(()=>{
this.loadCourses();
});
}
}
}

```

course-list.component.html

```

<h2>Course Management</h2>
<button (click)="addCourse()">Add Course</button>
<button (click)="updateCourse()">Update Course</button>
<table border="1">
<tr>
<th>ID</th>

```

```
<th>Name</th>
<th>Code</th>
<th>Credits</th>
<th>Faculty</th>
<th>Action</th>
</tr>
<tr *ngFor="let course of courses">

<td>{{course.id}}</td>
<td>{{course.name}}</td>
<td>{{course.code}}</td>
<td>{{course.credits}}</td>
<td>{{course.faculty}}</td>
<td>
<button (click)="deleteCourse(course.id)">
Delete
</button>
</td>
</tr>
</table>
```

db.json

```
{
  "courses":[
    {
      "id":1,
      "name":"Angular",
      "code":"ANG101",
```

```
"credits":4,  
"faculty":"Mr. Kumar"  
},  
{  
"id":2,  
"name":"React",  
"code":"RCT201",  
"credits":3,  
"faculty":"Mrs. Priya"  
},  
{  
"id":3,  
"name":"Spring Boot",  
"code":"SPR301",  
"credits":5,  
"faculty":"Mr. Raj"  
},  
{  
"id":4,  
"name":"Java",  
"code":"JAVA101",  
"credits":4,  
"faculty":"Mr. Arun"  
},  
{  
"id":5,  
"name":"MySQL",  
"code":"SQL101",
```

```
"credits":2,

"faculty":"Mrs. Kavya"

}

]

}
```

Start JSON Server

```
npx json-server --watch db.json --port 3000
```

Run Angular Application

```
ng serve
```

Open Browser

```
http://localhost:4200
```

Output Screenshot:

The screenshot displays the 'Student Course Portal' application interface and its underlying API interactions. The main page shows a table of courses with columns for ID, Course Name, Code, Credits, Faculty, and Actions. The initial list includes 6 courses: Angular, React, Spring Boot, Java, MySQL, and Python. The browser console shows logs for successful API calls, including GET /courses, POST /courses, PUT /courses/6, and DELETE /courses/6. The network tab shows the requests and responses for these operations, including the Authorization header added by the interceptor.

1. Initial Course List (Loaded using HTTP GET)

ID	Course Name	Code	Credits	Faculty	Actions
1	Angular	ANG101	4	Mr. Kumar	[Edit] [Delete]
2	React	RCT201	3	Mrs. Priya	[Edit] [Delete]
3	Spring Boot	SPR301	5	Mr. Arun	[Edit] [Delete]
4	Java	JAVA101	4	Ms. Divya	[Edit] [Delete]
5	MySQL	SQL101	2	Mr. Ramesh	[Edit] [Delete]
6	Python	PYT101	3	Dr. Neha	[Edit] [Delete] [New]

2. Add Course (POST) -> New Course Added

3. Update Course (PUT) -> Course Updated

4. Delete Course (DELETE) -> Course Removed

5. List Refreshed & Count Updated

6. Network Tab (Shows CRUD Requests)

7. Request Headers (Interceptor Adds Authorization Header)

8. Browser Console (tap() Logs for Successful API Calls)

After Each Operation

Key Outcomes Demonstrated

- GET: Courses retrieved successfully
- POST: Course added successfully
- PUT: Course updated successfully
- DELETE: Course removed successfully
- Interceptor adds Authorization header
- retry() retries failed requests up to 2 times
- tap() logs successful API calls
- Course list refreshes after every operation